
Institut Polytechnique de Saint-Louis

Département Informatique

Rapport Technique

Système de Gestion des Notes

(SGN)

Analyse Lexicale et Syntaxique avec
Flex, Bison & Interface Python

Module	Théorie des Langages & Compilation
Classe	GeIT 2
Année académique	2025–2026
Date de rendu	15 Juin 2026

Auteurs

Macky Mamadou WONE	Analyseur lexical, syntaxique, AST, table de symboles
Modou Mbaye	Interface graphique Python
Mouhammad Naasiri Diin DEME	Rapport technique

Table des matières

1	Introduction	1
1.1	Contexte et Motivation	1
1.2	Objectifs du Projet	1
1.3	Organisation du Travail	1
1.4	Structure du Document	1
2	Spécification du Langage .sgn	2
2.1	Description du Domaine	2
2.2	Structure d'un Fichier d'Entrée	2
2.3	Grammaire Formelle (BNF)	2
2.4	Tokens Lexicaux	3
3	Implémentation de l'Analyseur Lexical (Flex)	3
3.1	Principes Généraux	4
3.2	Définitions de Patterns	4
3.3	Règles Lexicales	4
3.4	Gestion des Erreurs Lexicales	5
3.5	Mode Test Autonome	5
4	Implémentation de l'Analyseur Syntaxique et Sémantique (Bison)	5
4.1	Architecture du Fichier parser.y	6
4.1.1	Variables Globales de Contexte	6
4.2	Règles Syntaxiques et Actions Sémantiques	6
4.2.1	Niveau	6
4.2.2	Étudiant	6
4.2.3	Module — Vérifications Sémantiques Immédiates	6
4.3	Vérifications Sémantiques	7
4.3.1	Gestion des Erreurs Syntaxiques	7
4.4	Règles de Calcul	7
4.4.1	Moyenne Pondérée d'un Semestre	7
4.4.2	Moyenne Annuelle	8
4.4.3	Décision de Passage et Mentions	8
4.4.4	Classement (Rang)	8
4.5	Sortie JSON	8
5	Structures de Données	9
5.1	Arbre Syntaxique Abstrait (AST)	9
5.2	Hierarchie de l'AST	10
5.3	Table de Symboles	10
5.4	Structures de Données Métier	11
6	Interface Graphique Python	11
6.1	Présentation Générale	11
6.2	Architecture des Fichiers Python	11
6.3	Communication Python ↔ C	11

6.4	Composants de l'Interface	12
6.4.1	Éditeur de Code (CodeEditor)	12
6.4.2	Tableau de Résultats (ResultsPanel)	12
6.4.3	Console Logs (ConsolePanel)	12
6.4.4	Barre de Navigation	12
7	Tests et Validation	13
7.1	Organisation des Tests	13
7.2	Test Valide — test_valide.sgn	13
7.2.1	Exemples de Calculs	13
7.3	Test Erreurs Lexicales — test_erreur_lex.sgn	13
7.4	Test Erreurs Syntaxiques — test_erreur_syn.sgn	14
7.5	Test Erreurs Sémantiques — test_erreur_sem.sgn	14
7.6	Compilation et Exécution	14
8	Conclusion	14
8.1	Bilan Technique	14
8.2	Difficultés Rencontrées	15
8.3	Perspectives d'Évolution	15
	Références Bibliographiques	15

1. Introduction

1.1. Contexte et Motivation

Dans le cadre du module de **Théorie des Langages et Compilation** de la formation GeIT 2 à l’Institut Polytechnique de Saint-Louis (IPSL), nous avons conçu et implémenté un **Système de Gestion des Notes** (SGN). Ce projet a pour objectif de mettre en pratique les concepts fondamentaux enseignés tout au long du semestre :

- La conception d’une **grammaire formelle** pour un langage dédié (DSL),
- L’implémentation d’un **analyseur lexical** avec l’outil `Flex`,
- L’implémentation d’un **analyseur syntaxique et sémantique** avec `Bison`,
- La construction d’une **représentation intermédiaire** (Arbre Syntaxique Abstrait + table de symboles),
- La réalisation d’une **interface graphique** Python reliant le frontal visuel au compilateur C.

1.2. Objectifs du Projet

Le système doit analyser des fichiers de notes étudiantes au format `.sgn` (format défini pour le projet), vérifier leur conformité syntaxique et sémantique, calculer automatiquement les moyennes, rangs et mentions, puis exporter ces résultats via une interface graphique.

À la fin de ce projet, nous sommes capables de :

- Définir et analyser un **langage dédié** (DSL),
- Interconnecter un **compilateur C** avec un **frontend Python**,
- Gérer proprement les **erreurs** à tous les niveaux (lexical, syntaxique, sémantique).

1.3. Organisation du Travail

Membre	Rôle principal	Fichiers
Macky Mamadou WONE	Compilateur C	<code>lexer.l</code> , <code>parser.y</code> , <code>ast.c/.h</code> , <code>symboles.c/</code>
Modou Mbaye	Interface Python	<code>app.py</code> , <code>interface.py</code> , <code>parser_bridge.py</code>
Mouhammad Naasiri Diin DEME	Rapport	<code>rapport.tex</code>

1.4. Structure du Document

Ce rapport est organisé comme suit : la section 2 définit le langage `.sgn` et sa grammaire ; la section 3 détaille l’analyseur lexical `Flex` ; la section 4 présente l’analyseur syntaxique et sémantique `Bison` ; la section 5 décrit les structures de données (AST et table de symboles) ; la section 6 présente l’interface Python ; la section 7 documente les tests et la validation ; enfin la section 8 conclut et ouvre des perspectives.

2. Spécification du Langage .sgn

2.1. Description du Domaine

Le langage .sgn est un **DSL** (Domain-Specific Language) structuré pour décrire les notes d'étudiants répartis sur trois niveaux d'études :

Niveau	Désignation	Modules types	Semestres
L1	Première année de Licence	Maths, Algo, Réseaux, Physique	S1, S2
L2	Deuxième année de Licence	POO, BDD, Systèmes, Stats	S3, S4
L3	Troisième année de Licence	Compilation, IA, Génie Logiciel	S5, S6

Chaque étudiant possède un **matricule unique**, un nom complet, et une liste de notes par module avec un coefficient associé.

2.2. Structure d'un Fichier d'Entrée

Un fichier .sgn suit une organisation hiérarchique stricte : le fichier commence obligatoirement par la déclaration de l'année académique, puis liste les niveaux, chacun contenant des étudiants avec leurs semestres et modules.

```

1 -- Fichier notes : Annee academique 2025-2026
2 ANNEE "2025-2026"
3
4 NIVEAU L1 {
5   ETUDIANT {
6     MATRICULE : "L1-2025-001"
7     NOM : "DIALLO Mamadou"
8     PRENOM : "Mamadou"
9     SEMESTRE S1 {
10      MODULE "Algorithmique" COEF 3 NOTE 14.5
11      MODULE "Mathematiques" COEF 4 NOTE 12.0
12      MODULE "Architecture" COEF 2 NOTE 16.0
13      MODULE "Anglais" COEF 1 NOTE 15.0
14    }
15    SEMESTRE S2 {
16      MODULE "Programmation C" COEF 3 NOTE 13.5
17      MODULE "Logique" COEF 3 NOTE 11.0
18      MODULE "Reseaux" COEF 2 NOTE 17.0
19    }
20  }
21 }
```

Listing 1: Extrait d'un fichier .sgn valide

Les lignes commençant par – sont des **commentaires** et sont ignorées par l'analyseur.

2.3. Grammaire Formelle (BNF)

La grammaire complète du langage .sgn est définie ci-dessous :

```

1 <programme> ::= ANNEE STRING <liste_niveaux>
```

```

2 <liste_niveaux> ::= <niveau> | <liste_niveaux> <niveau>
3 <niveau> ::= NIVEAU <id_niveau> '{' <liste_etudiants> '}'
4 <id_niveau> ::= L1 | L2 | L3
5 <liste_etudiants> ::= <etudiant> | <liste_etudiants> <etudiant>
6 <etudiant> ::= ETUDIANT '{' <champs_etudiant> <
    liste_semestres> '}'
7 <champs_etudiant> ::= MATRICULE ':' STRING
8                       NOM ':' STRING
9                       PRENOM ':' STRING
10 <liste_semestres> ::= <semestre> | <liste_semestres> <semestre>
11 <semestre> ::= SEMESTRE <id_sem> '{' <liste_modules> '}'
12 <id_sem> ::= S1 | S2 | S3 | S4 | S5 | S6
13 <liste_modules> ::= <module> | <liste_modules> <module>
14 <module> ::= MODULE STRING COEF ENTIER NOTE REEL
15 <STRING> ::= '"' [^"]* '"'
16 <REEL> ::= [0-9]+ ('.' [0-9]+)?
17 <ENTIER> ::= [0-9]+

```

Listing 2: Grammaire BNF du langage .sgn

La grammaire est de type **LL(1)** compatible, ce qui la rend propice à une implémentation avec Bison (analyseur LALR(1)).

2.4. Tokens Lexicaux

Le tableau suivant recense l'ensemble des tokens reconnus par l'analyseur lexical :

Token	Description	Valeur associée	Exemple
ANNEE	Mot-clé de début	—	ANNEE
NIVEAU	Déclaration niveau	—	NIVEAU
ETUDIANT	Bloc étudiant	—	ETUDIANT
MATRICULE	Identifiant	—	MATRICULE
NOM	Champ nom	—	NOM
PRENOM	Champ prénom	—	PRENOM
SEMESTRE	Décl. semestre	—	SEMESTRE
MODULE	Décl. module	—	MODULE
COEF	Coefficient	—	COEF
NOTE	Note obtenue	—	NOTE
L1/L2/L3	Id. de niveau	—	L1
S1..S6	Id. de semestre	—	S3
REEL	Nombre réel	double	14.5
ENTIER	Nombre entier	int	3
STRING	Chaîne entre guillemets	char*	"Algorithmique"
LBRACE	Accolade ouvrante	—	{
RBRACE	Accolade fermante	—	}
COLON	Deux-points	—	:
ERREUR_LEX	Caractère invalide	—	@, #, \$

3. Implémentation de l'Analyseur Lexical (Flex)

3.1. Principes Généraux

L'analyseur lexical est implémenté dans le fichier `src/lexer.l`. Il constitue le premier maillon de la chaîne de compilation : il lit le flux de caractères du fichier `.sgn` et le découpe en **tokens** significatifs qui sont transmis à l'analyseur syntaxique Bison.

Flex génère automatiquement le fichier `lex.yy.c` à partir de `lexer.l`. Les deux options Flex essentielles utilisées sont :

- `%option noyywrap` — pas de fichier multiple à enchaîner,
- `%option yylineno` — comptage automatique des numéros de ligne pour les messages d'erreur.

3.2. Définitions de Patterns

Les macros suivantes simplifient l'écriture des règles :

```
1 CHIFFRE      [0-9]
2 BLANC        [ \t\r]+
3 REEL_DEF     {CHIFFRE}+("."{CHIFFRE}+)?
4 STRING_DEF   \"[^\n]*\"
```

Listing 3: Macros de patterns dans `lexer.l`

Le pattern `REEL_DEF` reconnaît à la fois les entiers (14) et les réels (14.5). La distinction entre `ENTIER` et `REEL` est faite en vérifiant la présence d'un point décimal via `strchr()`.

3.3. Règles Lexicales

```
1  "--" [^\n]*      { /* commentaire : ignorer */ }
2
3  "ANNEE"           { return ANNEE;      }
4  "NIVEAU"          { return NIVEAU_KW;   }
5  "ETUDIANT"        { return ETUDIANT;    }
6  "MATRICULE"       { return MATRICULE;   }
7  "NOM"             { return NOM_TOK;     }
8  "PRENOM"          { return PRENOM_TOK;  }
9  "SEMESTRE"        { return SEMESTRE;    }
10 "MODULE"          { return MODULE_KW;   }
11 "COEF"            { return COEF;        }
12 "NOTE"            { return NOTE;        }
13
14 "L1"               { return L1;         }
15 "L2"               { return L2;         }
16 "L3"               { return L3;         }
17 "S1"               { return S1;         } /* ... S2..S6 idem */
18
19 {REEL_DEF}         {
20     if (strchr(yytext, '.') != NULL) {
21         yylval.reel = atof(yytext);
22         return REEL;
23     } else {
24         yylval.entier = atoi(yytext);
25         return ENTIER;
26     }
27 }
```

```

27 }
28
29 {STRING_DEF} {
30     int len = yyleng - 2;
31     yylval.chaine = malloc(len + 1);
32     strncpy(yylval.chaine, yytext + 1, len);
33     yylval.chaine[len] = '\0';
34     return STRING;
35 }
36
37 {BLANC}          { /* ignorer espaces et tabulations */ }
38 "\n"            { /* saut de ligne gere par yylineno */ }
39
40 .               {
41     /* Caractere non reconnu -> erreur lexicale */
42     char msg[64];
43     snprintf(msg, sizeof(msg),
44              "Caractere inattendu : '%s'", yytext);
45     erreur_lexicale(msg, yylineno);
46     return ERREUR_LEX;
47 }

```

Listing 4: Extrait des règles de reconnaissance dans lexer.l

Ordre de priorité : Flex applique les règles dans l'ordre de leur déclaration en cas d'ambiguïté. Les mots-clés sont donc listés *avant* la règle générique `.` (catch-all), garantissant leur reconnaissance prioritaire.

3.4. Gestion des Erreurs Lexicales

Tout caractère non reconnu par les patterns précédents déclenche la règle `.` et appelle `erreur_lexicale()` :

```

1 void erreur_lexicale(const char *msg, int ligne) {
2     fprintf(stderr, "[ERREUR LEXICALE] Ligne %d : %s\n",
3             ligne, msg);
4 }

```

Listing 5: Fonction de signalement d'erreur lexicale

Le message est écrit sur `stderr` (séparé du flux JSON sur `stdout`), ce qui permet à Python de les capturer indépendamment.

3.5. Mode Test Autonome

Le fichier `lexer.l` intègre un mode test conditionnel (`#ifdef TEST_LEXER_ONLY`) permettant de le compiler seul et de vérifier la liste de tokens produits avant même d'intégrer Bison.

4. Implémentation de l'Analyseur Syntaxique et Sémantique (Bison)

4.1. Architecture du Fichier `parser.y`

Le fichier `src/parser.y` est divisé en quatre sections canoniques :

1. **Déclarations C** — includes, variables globales de contexte,
2. **Déclarations Bison** — union sémantique, liste des tokens et types,
3. **Règles de grammaire** — productions BNF avec actions sémantiques,
4. **Fonctions C** — `yyerror`, fonctions de calcul et de génération JSON.

4.1.1. Variables Globales de Contexte

```

1 Programme      g_programme;           /* toutes les donnees parsees
   */
2 TableSymboles  g_table;               /* detection des doublons
   */
3 ASTNode        *g_ast_racine = NULL;  /* racine de l'AST
   */
4 int            g_erreurs = 0;          /* compteur global d'erreurs
   */
5
6 Niveau         *g_niveau_courant = NULL;
7 Etudiant       *g_etudiant_courant = NULL;
8 Semestre       *g_semestre_courant = NULL;
```

Listing 6: Variables de contexte dans `parser.y`

Ces pointeurs de contexte sont mis à jour au fil du parsing pour éviter une navigation coûteuse dans les structures imbriquées.

4.2. Règles Syntaxiques et Actions Sémantiques

Chaque règle de production est associée à un bloc d'action C exécuté lors de la réduction. Voici les points clés :

4.2.1. Niveau

À l'ouverture d'un bloc NIVEAU, un slot est réservé dans `g_programme.niveaux[]` et un nœud AST de type `NODE_NIVEAU` est créé et rattaché à la racine. En fin de bloc, la fonction `calculer_rangs()` trie et attribue les rangs aux étudiants du niveau.

4.2.2. Étudiant

À l'ouverture d'un bloc ETUDIANT, un slot est réservé et un nœud `NODE_ETUDIANT` est créé. À la fermeture du bloc, la moyenne annuelle et la mention sont calculées automatiquement.

4.2.3. Module — Vérifications Sémantiques Immédiates

```

1 module
2     : MODULE_KW STRING COEF ENTIER NOTE REEL
3     {
4         verifier_note($6, yylineno);  /* note dans [0,20] */
```

```

5         verifier_coef($4, yylineno);  /* coef >= 1          */
6
7         int idx = g_semestre_courant->nb_modules++;
8         strncpy(g_semestre_courant->modules[idx].nom, $2, ...);
9         g_semestre_courant->modules[idx].coef = $4;
10        g_semestre_courant->modules[idx].note = $6;
11
12        astajouter_enfant(g_ast_semestre_courant,
13                          ast_creer_module($2, $4, $6, yylineno));
14        free($2);
15    }
16    ;

```

Listing 7: Action sémantique d'un module

4.3. Vérifications Sémantiques

Quatre règles sémantiques sont appliquées :

N°	Règle	Condition	Message d'erreur
1	Note valide	$0.0 \leq \text{note} \leq 20.0$	[ERREUR SEMANTIQUE] Note hors
2	Coefficient	$\text{coef} \geq 1$	[ERREUR SEMANTIQUE] Coefficient
3	Semestre/Niveau	$L1 \rightarrow S1, S2 ; L2 \rightarrow S3, S4 ; L3 \rightarrow S5, S6$	[ERREUR SEMANTIQUE] Semestre
4	Unicité matricule	Pas de doublon dans un niveau	[ERREUR SEMANTIQUE] Matricule

4.3.1. Gestion des Erreurs Syntaxiques

La fonction `yyerror()` est automatiquement appelée par Bison lors d'une erreur syntaxique :

```

1 void yyerror(const char *msg) {
2     fprintf(stderr, "[ERREUR SYNTAXIQUE] Ligne %d : %s\n",
3             yylineno, msg);
4     g_erreurs++;
5 }

```

Listing 8: Fonction yyerror

4.4. Règles de Calcul

4.4.1. Moyenne Pondérée d'un Semestre

$$\overline{M}_s = \frac{\sum_{i=1}^n c_i \times n_i}{\sum_{i=1}^n c_i} \quad (1)$$

où c_i est le coefficient du module i et n_i sa note.

4.4.2. Moyenne Annuelle

$$\overline{M}_a = \frac{1}{S} \sum_{s=1}^S \overline{M}_s \quad (2)$$

S est le nombre de semestres validés par l'étudiant.

4.4.3. Décision de Passage et Mentions

Intervalle	Décision	Mention
$\overline{M}_a \geq 16,0$	Admis	Très Bien
$14,0 \leq \overline{M}_a < 16,0$	Admis	Bien
$12,0 \leq \overline{M}_a < 14,0$	Admis	Assez Bien
$10,0 \leq \overline{M}_a < 12,0$	Admis	Passable
$\overline{M}_a < 10,0$	Ajourné	—

4.4.4. Classement (Rang)

Le rang est calculé par un algorithme de comparaison croisée : pour chaque étudiant i , son rang est le nombre d'étudiants $j \neq i$ ayant une moyenne annuelle strictement supérieure, plus un.

```

1 void calculer_rangs(Niveau *niv) {
2     int i, j, n = niv->nb_etudiants;
3     for (i = 0; i < n; i++) niv->etudiants[i].rang = 1;
4     for (i = 0; i < n; i++)
5         for (j = 0; j < n; j++)
6             if (i != j &&
7                 niv->etudiants[j].moyenne_annuelle >
8                 niv->etudiants[i].moyenne_annuelle)
9                 niv->etudiants[i].rang++;
10 }
```

Listing 9: Algorithme de calcul des rangs

4.5. Sortie JSON

Une fois le parsing terminé, la fonction `afficher_json()` écrit sur `stdout` un objet JSON structuré, lisible directement par Python :

```

1 {
2     "annee": "2025-2026",
3     "niveaux": [
4         {
5             "niveau": "L1",
6             "etudiants": [
7                 {
8                     "matricule": "L1-2025-001",
9                     "nom": "DIALLO Mamadou",
10                    "prenom": "Mamadou",
```

```

11         "semestres": [
12             {
13                 "id": "S1",
14                 "modules": [
15                     {"nom": "Algorithmique", "coef": 3, "note": 14.50}
16                 ],
17                 "moyenne": 13.80
18             }
19         ],
20         "moyenne_annuelle": 13.80,
21         "mention": "Assez Bien",
22         "decision": "Admis",
23         "rang": 1
24     }
25 ]
26 }
27 ],
28 "erreurs": [],
29 "nb_erreurs": 0
30 }

```

Listing 10: Format de sortie JSON du binaire SGN

Les erreurs (lexicales, syntaxiques, sémantiques) sont écrites sur **stderr**, séparément du JSON :

```

[ERREUR LEXICALE]   Ligne 12 : Caractere inattendu : '@'
[ERREUR SYNTAXIQUE] Ligne 18 : syntax error
[ERREUR SEMANTIQUE] Ligne 23 : Note 25.00 hors intervalle [0.0, 20.0]

```

5. Structures de Données

5.1. Arbre Syntaxique Abstrait (AST)

Nous avons implémenté un AST générique à enfants dynamiques. Chaque nœud est représenté par la structure `ASTNode` définie dans `ast.h` :

```

1  typedef enum {
2      NODE_PROGRAMME,      /* racine */
3      NODE_NIVEAU,         /* L1, L2 ou L3 */
4      NODE_ETUDIANT,       /* bloc etudiant */
5      NODE_MATRICULE,      /* feuille : matricule */
6      NODE_NOM,            /* feuille : nom */
7      NODE_PRENOM,         /* feuille : prenom */
8      NODE_SEMESTRE,       /* S1..S6 */
9      NODE_MODULE,         /* module avec coef et note */
10     NODE_ANNEE            /* valeur de l'annee academique */
11 } NodeType;
12
13 typedef struct ASTNode {
14     NodeType type;
15     NodeValue valeur;      /* union : chaine / entier / reel */
16     int coef;              /* pour NODE_MODULE */
17     double note;           /* pour NODE_MODULE */

```

```

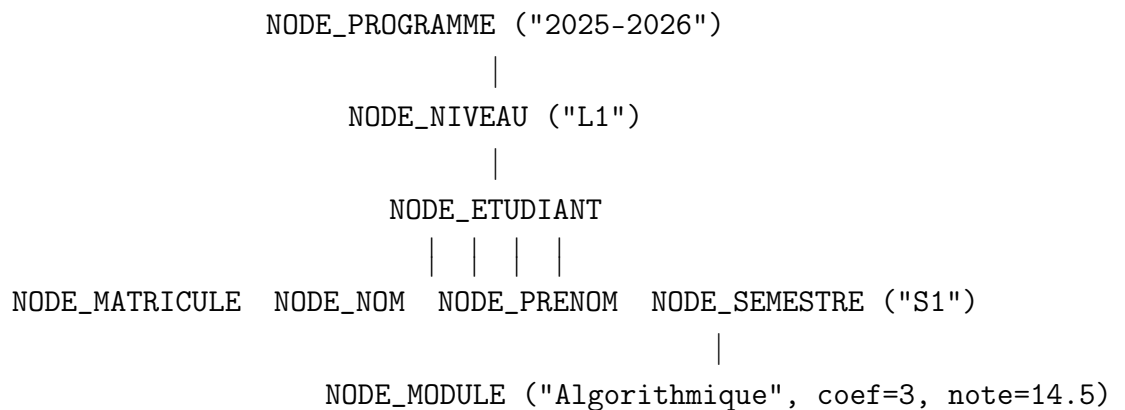
18     struct ASTNode **enfants;
19     int             nb_enfants;
20     int             capacite;
21     int             ligne;
22 } ASTNode;

```

Listing 11: Structure d'un nœud AST dans ast.h

5.2. Hiérarchie de l'AST

La figure suivante illustre la structure arborescente construite par le parseur :



L'AST sert principalement à :

1. Conserver toutes les informations en mémoire pour une analyse globale,
2. Calculer le rang (qui nécessite de connaître *tous* les étudiants d'un niveau),
3. Générer le flux JSON en parcourant l'arbre en fin de traitement.

5.3. Table de Symboles

La table de symboles sert à détecter les doublons de matricule au sein d'un même niveau. Elle est implémentée comme un tableau de 600 entrées ($\text{MAX_NIVEAUX} \times \text{MAX_ETUDIANTS} = 3 \times 200$).

```

1  typedef struct {
2      char matricule[64];    /* cle : matricule */
3      char niveau_id[4];    /* niveau d'appartenance */
4      int  ligne;           /* ligne de declaration */
5      int  occupe;          /* 1 si utilise, 0 sinon */
6  } EntreeSymbole;
7
8  typedef struct {
9      EntreeSymbole entrees[TAILLE_TABLE];
10     int            nb_entrees;
11 } TableSymboles;

```

Listing 12: Structure de la table de symboles dans symboles.h

La fonction `symb_inserer()` vérifie si un matricule est déjà présent dans la table avant d'insérer ; en cas de doublon, elle écrit un message sur `stderr` et incrémente

le compteur d'erreurs.

5.4. Structures de Données Métier

Les structures de données principales, définies dans `symboles.h`, reflètent directement la hiérarchie du langage :

Structure	Contenu
Module	<code>nom[128]</code> , <code>coef</code> , <code>note</code>
Semestre	<code>id[4]</code> , <code>modules[50]</code> , <code>nb_modules</code> , <code>moyenne</code>
Etudiant	<code>matricule[64]</code> , <code>nom[128]</code> , <code>prenom[64]</code> , <code>semestres[6]</code> , <code>moyenne_annuelle</code> , <code>mention</code>
Niveau	<code>id[4]</code> , <code>etudiants[200]</code> , <code>nb_etudiants</code>
Programme	<code>annee[16]</code> , <code>niveaux[3]</code> , <code>nb_niveaux</code>

6. Interface Graphique Python

6.1. Présentation Générale

L'interface graphique est développée en **Python 3** avec la bibliothèque **PyQt5**. Elle offre un environnement de travail complet pour charger, éditer, analyser et exporter les fichiers `.sgn`.

L'application suit le modèle **MVC** (Modèle-Vue-Contrôleur) :

- **Modèle** — données JSON retournées par le binaire C,
- **Vue** — widgets Qt (éditeur, tableau, console) définis dans `interface.py`,
- **Contrôleur** — méthodes de `SGNAnalyzer` dans `interface.py` et pont dans `parser_bridge.py`.

6.2. Architecture des Fichiers Python

Fichier	Rôle
<code>app.py</code>	Point d'entrée ; initialise <code>QApplication</code> et applique le style Fusion
<code>interface.py</code>	Définit <code>SGNAnalyzer</code> , <code>CodeEditor</code> , <code>ResultsPanel</code> , <code>ConsolePanel</code> , <code>SGNHighlighter</code>
<code>parser_bridge.py</code>	Exécute le binaire C via <code>subprocess</code> , capture <code>stdout/stderr</code> et retourne le dic

6.3. Communication Python ↔ C

La communication est réalisée exclusivement via `subprocess.run()` :

```
1 import subprocess, json
2
3 def run_parser(filepath):
4     result = subprocess.run(
5         ["./src/sgn", filepath],
6         capture_output=True,
```

```

7         text=True
8     )
9     data = json.loads(result.stdout)
10    errors = result.stderr.strip().splitlines()
11    data["erreurs"] = errors if errors else []
12    return data

```

Listing 13: Pont Python–C dans parser_bridge.py

- **stdout** → JSON structuré (résultats d'analyse),
- **stderr** → messages d'erreurs (lexicales / syntaxiques / sémantiques),
- **returncode** → 0 si succès, 1 si erreurs détectées.

6.4. Composants de l'Interface

6.4.1. Éditeur de Code (CodeEditor)

Zone de saisie et d'édition du fichier .sgn avec :

- Coloration syntaxique via `SGNHighlighter` (`QSyntaxHighlighter`),
- Barre de titre de style éditeur moderne (points rouge/jaune/vert),
- Police monospace Consolas 12pt, fond sombre #1a1e2e.

6.4.2. Tableau de Résultats (ResultsPanel)

Tableau `QTableWidget` avec les colonnes : *Matricule*, *Nom & Prénom*, *Niveau*, *Moy. Semestre(s)*, *Moy. Annuelle*, *Rang*, *Mention*, *Décision*.

La mention et la décision sont affichées avec des *badges* colorés dynamiques (vert pour « Admis », rouge pour « Ajourné », bleu/vert selon le niveau de mention).

6.4.3. Console Logs (ConsolePanel)

Panneau d'affichage des erreurs retournées par le binaire. En cas d'analyse réussie, un message de succès en vert est affiché. Les erreurs sont listées avec une icône d'erreur (X) et affichées en rouge.

6.4.4. Barre de Navigation

La barre supérieure propose quatre actions :

1. **Upload .sgn** — sélection d'un fichier via dialogue,
2. **Save** — sauvegarde du fichier en cours d'édition,
3. **Run Analysis** — déclenchement de l'analyse complète,
4. **Export CSV** — export des résultats au format CSV.

7. Tests et Validation

7.1. Organisation des Tests

Quatre fichiers de test sont fournis dans le dossier `tests/` :

Fichier	Contenu
<code>test_valide.sgn</code>	3 niveaux, 6 étudiants, toutes les mentions
<code>testErreurLex.sgn</code>	Caractères @, #, \$, guillemet non fermé
<code>testErreurSyn.sgn</code>	PRENOM manquant, accolade oubliée, NOTE absent, MATRICULE absent
<code>testErreurSem.sgn</code>	Coef=0, note=25.0, note=-3.0, S4 dans L1, doublon matricule

7.2. Test Valide — `test_valide.sgn`

Le fichier contient 3 niveaux (L1 : 3 étudiants, L2 : 2 étudiants, L3 : 1 étudiant) avec toutes les mentions couvertes. Le binaire ne produit aucune erreur sur `stderr` et le JSON est syntaxiquement valide.

Commande de test : `./sgn ../tests/test_valide.sgn | python3 -m json.tool`

7.2.1. Exemples de Calculs

Pour l'étudiant L1-2025-001 - DIALLO Mamadou, le calcul de la moyenne du semestre S1 est :

$$\overline{M}_{S1} = \frac{3 \times 14,5 + 4 \times 12,0 + 2 \times 16,0 + 1 \times 15,0}{3 + 4 + 2 + 1} = \frac{43,5 + 48 + 32 + 15}{10} = \frac{138,5}{10} = 13,85$$

7.3. Test Erreurs Lexicales — `test_erreur_lex.sgn`

Ce fichier introduit volontairement des caractères non reconnus :

Erreur	Ligne	Description
1	12	Caractère @ dans la valeur de note (14@5)
2	18	Guillemet non fermé dans le champ NOM
3	24	Caractère # en début d'identifiant
4	34	Caractère \$ dans la valeur de coefficient

Le binaire produit sur `stderr` :

```
[ERREUR LEXICALE] Ligne 12 : Caractere inattendu : '@'
[ERREUR LEXICALE] Ligne 18 : Caractere inattendu : (guillemet)
[ERREUR LEXICALE] Ligne 24 : Caractere inattendu : '#'
[ERREUR LEXICALE] Ligne 34 : Caractere inattendu : '$'
```

7.4. Test Erreurs Syntaxiques — `test_erreur_syn.sgn`

Erreur	Ligne	Description
1	10	PRENOM manquant (SEMESTRE S1 arrive avant PRENOM)
2	23	Accolade fermante de SEMESTRE S1 oubliée
3	33	Mot-clé NOTE absent avant la valeur numérique
4	38	MATRICULE absent en début de bloc ETUDIANT

7.5. Test Erreurs Sémantiques — `test_erreur_sem.sgn`

Erreur	Ligne	Description
1	13	Coefficient 0 invalide (COEF 0 NOTE 14.5)
2	15	Note 25.0 hors intervalle [0.0, 20.0]
3	17	Note -3.0 hors intervalle [0.0, 20.0]
4	21	Semestre S4 incompatible avec le niveau L1
5	27	Doublon de matricule L1-2025-030 dans le niveau L1

7.6. Compilation et Exécution

```
cd src/
make                # genere le binaire sgn
make test           # test_valide.sgn -> 0 erreur
make test_lex       # test_erreur_lex.sgn -> 4 erreurs
    lexicales
make test_err       # test_erreur_sem.sgn -> 5 erreurs
    semantiques
```

Listing 14: Commandes de compilation et de test

La chaîne de compilation exécutée par `make` est :

1. `bison -d parser.y → parser.tab.c, parser.tab.h`
2. `flex lexer.l → lex.yy.c`
3. Compilation séparée de chaque fichier `.c` en `.o`
4. Édition de liens finale → binaire `sgn` (avec `-lm` pour les maths)

8. Conclusion

8.1. Bilan Technique

Ce projet nous a permis de maîtriser l'ensemble de la chaîne de compilation : de la définition d'un langage formel jusqu'à son intégration dans une application graphique complète.

Points forts de l'implémentation :

- L'**analyseur lexical** reconnaît tous les tokens du langage et signale précisément les erreurs avec le numéro de ligne.
- L'**analyseur syntaxique** valide la structure hiérarchique et la **table de symboles** détecte les doublons de matricule.
- Les **vérifications sémantiques** couvrent les quatre cas demandés (notes, coefficients, semestres, doublons).
- L'**AST** permet de calculer les rangs après avoir parsé l'ensemble d'un niveau.
- La **sortie JSON** sur `stdout` et les **erreurs sur `stderr`** constituent une interface propre et découplée entre le compilateur C et le frontend Python.

8.2. Difficultés Rencontrées

- **Gestion mémoire en C** : la construction du JSON sans bibliothèque tierce a nécessité l'écriture d'une fonction d'échappement (`json_string_safe()`) pour traiter les caractères spéciaux.
- **Synchronisation `stdout/stderr`** : s'assurer que Python capture correctement les deux flux séparément via `subprocess.run(..., capture_output=True)`.
- **Règles de priorité Flex** : l'ordre des règles est critique pour que les mots-clés soient reconnus avant la règle catch-all ..
- **Gestion des contextes Bison** : l'utilisation de pointeurs globaux de contexte (niveau, étudiant, semestre courants) a simplifié considérablement la gestion des structures imbriquées.

8.3. Perspectives d'Évolution

Le système est aisément extensible :

- **Nouveaux champs** : ajout de crédits ECTS, d'UE (Unités d'Enseignement), ou de modules d'option sans modifier la structure de base.
- **Export PDF** : génération directe d'un relevé de notes depuis l'interface Python.
- **Support multi-fichiers** : fusion de plusieurs fichiers `.sgn` en un rapport global.
- **Récupération d'erreur Bison** : l'implémentation du token `error` de Bison permettrait de continuer l'analyse après une erreur syntaxique.

Références Bibliographiques

1. John Levine, *Flex & Bison*, O'Reilly Media, 2009.
2. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools* (2^{ème} édition), Pearson, 2006.
3. Documentation officielle Flex : <https://github.com/westes/flex>
4. Documentation officielle Bison : <https://www.gnu.org/software/bison/manual/>

5. Documentation Python subprocess : <https://docs.python.org/3/library/subprocess.html>
6. Documentation PyQt5 : <https://www.riverbankcomputing.com/software/pyqt/>